ISE 291 – Project Milestone 2: Data Cleaning

Project: Telco Customer Churn

Team Members (Name – ID – Contribution):

- Majed AlOwayyeidh (202467820) - Loaded the Telco churn dataset, coordinated Milestone 2 workflow, prepared the notebook structure, and documented the main cleaning plan (handling TotalCharges missing/blank values, type conversions, and final export).
- Talal AlDakheel (202467400) - Performed data quality checks (missing values summary + consistency checks), validated key churn-related fields for analysis readiness, and wrote the "issues found + actions taken" explanation section for Milestone 2.
- Ahmed AlSaad (202432980) - Identified variable types (categorical vs numerical), verified mixed data types requirement, and implemented preprocessing steps (e.g., converting Yes/No columns and SeniorCitizen to appropriate types for analysis).
- Abdullah Alluqmani (202257720) - Investigated dataset context and meaning of variables, checked for unrealistic values/outliers in numeric columns (tenure, MonthlyCharges, TotalCharges), and contributed written interpretation of cleaning decisions (why imputation was needed).
- Rakan AlShalan (202351430) - Ensured Milestone 2 submission meets formatting and rubric requirements, verified that all required outputs are included (Jupyter + Excel), reviewed notebook readability, and finalized the cleaned dataset export file naming/structure.

```
import pandas as pd
import numpy as np

pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 140)
```

```
# Load the raw data

df = pd.read_csv("/content/WA_Fn-UseC_-Telco-Customer-Churn.csv")  # Colab path after upload
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecur
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

```
# Basic overview
print("Shape:", df.shape)
display(df.info())
display(df.describe(include='all').T.head(30))
```

```
Shape: (7043, 21)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
customerID	7043	7043	3186-AJIEK	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612	0.0	0.0	0.0	0.0	1.0
Partner	7043	2	No	3641	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Dependents	7043	2	No	4933	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481	0.0	9.0	29.0	55.0	72.0
PhoneService	7043	2	Yes	6361	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN	NaN	NaN	NaN	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN	NaN	NaN	NaN	NaN	NaN
DeviceProtection	7043	3	No	3095	NaN	NaN	NaN	NaN	NaN	NaN	NaN
TechSupport	7043	3	No	3473	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	7043	2	Yes	4171	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047	18.25	35.5	70.35	89.85	118.75
TotalCharges	7043	6531		11	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Churn	7043	2	No	5174	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
# Variable list + types + unique values
var_types = pd.DataFrame({
    "variable": df.columns,
    "dtype": df.dtypes.astype(str).values,
    "n_unique": [df[c].nunique(dropna=True) for c in df.columns]
})
var_types
```

	variable	dtype	n_unique
0	customerID	object	7043
1	gender	object	2
2	SeniorCitizen	int64	2
3	Partner	object	2
4	Dependents	object	2
5	tenure	int64	73
6	PhoneService	object	2
7	MultipleLines	object	3
8	InternetService	object	3
9	OnlineSecurity	object	3
10	OnlineBackup	object	3
11	DeviceProtection	object	3
12	TechSupport	object	3
13	StreamingTV	object	3
14	StreamingMovies	object	3
15	Contract	object	3
16	PaperlessBilling	object	2
17	PaymentMethod	object	4
18	MonthlyCharges	float64	1585
19	TotalCharges	object	6531
20	Churn	object	2

Next steps: Generate code with var_types New interactive sheet

```
# Short descriptions (data dictionary)
# Keep it short but clear. Add more rows if you want.
data_dictionary = pd.DataFrame([
    {"variable":"customerID", "type":"categorical (ID)", "description":"Unique customer identifier"},
    {"variable":"gender", "type":"categorical", "description":"Customer gender"},
    {"variable":"SeniorCitizen", "type":"binary/categorical", "description":"1 if senior citizen, else 0"},
    {"variable":"Partner", "type":"categorical", "description":"Whether customer has a partner (Yes/No)"},
    {"variable":"Dependents", "type":"categorical", "description":"Whether customer has dependents (Yes/No)"},
    {"variable":"tenure", "type":"numeric", "description":"Months the customer has stayed with the company"},
    {"variable":"PhoneService", "type":"categorical", "description":"Whether customer has phone service (Yes/No)"},
    {"variable":"InternetService", "type":"categorical", "description":"Type of internet service (DSL/Fiber/No)"},
    {"variable":"Contract", "type":"categorical", "description":"Contract type (Month-to-month/One year/Two year)"},
    {"variable":"MonthlyCharges", "type":"numeric", "description":"Monthly charges billed to the customer"},
    {"variable":"TotalCharges", "type":"numeric (after cleaning)", "description":"Total amount charged to the customer"},
    {"variable":"Churn", "type":"target (categorical)", "description":"Whether customer churned (Yes/No)"},
])

# Merge with actual columns to avoid errors if any column names differ
data_dictionary = data_dictionary[data_dictionary["variable"].isin(df.columns)]
data_dictionary
```

	variable	type	description
0	customerID	categorical (ID)	Unique customer identifier
1	gender	categorical	Customer gender
2	SeniorCitizen	binary/categorical	1 if senior citizen, else 0
3	Partner	categorical	Whether customer has a partner (Yes/No)
4	Dependents	categorical	Whether customer has dependents (Yes/No)
5	tenure	numeric	Months the customer has stayed with the company
6	PhoneService	categorical	Whether customer has phone service (Yes/No)
7	InternetService	categorical	Type of internet service (DSL/Fiber/No)
8	Contract	categorical	Contract type (Month-to-month/One year/Two year)
9	MonthlyCharges	numeric	Monthly charges billed to the customer
10	TotalCharges	numeric (after cleaning)	Total amount charged to the customer
11	Churn	target (categorical)	Whether customer churned (Yes/No)

Next steps: Generate code with `data_dictionary` New interactive sheet

```
# Make a working copy and standardize strings
df2 = df.copy()
df2.columns = [c.strip() for c in df2.columns]

for c in df2.select_dtypes(include=['object']).columns:
    df2[c] = df2[c].astype(str).str.strip()

# Convert TotalCharges: blanks -> NaN -> numeric
df2['TotalCharges'] = df2['TotalCharges'].replace({'': np.nan, 'nan': np.nan, 'NaN': np.nan})
df2['TotalCharges'] = pd.to_numeric(df2['TotalCharges'], errors='coerce')

# Missing summary
missing = df2.isna().sum().sort_values(ascending=False)
missing = missing[missing > 0]
missing.to_frame('missing_count')
```

	missing_count
TotalCharges	11

```
# Inconsistency checks (service-related categories)
internet_cols = ["OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]

# For customers with no internet service, these should be "No internet service"
mask_no_internet = (df2["InternetService"] == "No")
df2.loc[mask_no_internet, internet_cols].apply(lambda s: (s == "No internet service").value_counts())
```

	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
True	1526	1526	1526	1526	1526	1526

```
# For customers with no phone service, MultipleLines should be "No phone service"
mask_no_phone = (df2["PhoneService"] == "No")
(df2.loc[mask_no_phone, "MultipleLines"] == "No phone service").value_counts()
```

MultipleLines	count
True	682

dtype: int64

```
# Outlier check (IQR) for key numeric variables
def iqr_outlier_count(series):
    s = pd.to_numeric(series, errors='coerce').dropna()
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    iqr = q3 - q1
    lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
    outliers = ((s < lo) | (s > hi)).sum()
    return int(outliers), float(lo), float(hi)

for col in ["tenure","MonthlyCharges","TotalCharges"]:
    cnt, lo, hi = iqr_outlier_count(df2[col])
```

```
    print(f"{col}: outliers={cnt}, bounds=({lo:.3f}, {hi:.3f})")
```

```
tenure: outliers=0, bounds=(-60.000, 124.000)
MonthlyCharges: outliers=0, bounds=(-46.025, 171.375)
TotalCharges: outliers=0, bounds=(-4688.481, 8884.669)
```

```
# Cleaning / preprocessing
clean = df2.copy()

# Handle missing TotalCharges:
# Typically missing when tenure == 0, so set to 0; otherwise impute median within Contract (fallback overall median)
mask_missing = clean['TotalCharges'].isna()
mask_tenure0 = (clean['tenure'].fillna(0).astype(float) == 0)

clean.loc[mask_missing & mask_tenure0, 'TotalCharges'] = 0.0

remaining = clean['TotalCharges'].isna()
if remaining.any():
    clean.loc[remaining, 'TotalCharges'] = clean.groupby('Contract')['TotalCharges'].transform('median')[remaining]
    clean['TotalCharges'] = clean['TotalCharges'].fillna(clean['TotalCharges'].median())

# Convert SeniorCitizen to category for reporting
clean['SeniorCitizen'] = clean['SeniorCitizen'].astype(int).astype('category')

# Convert Yes/No columns to category (keeps notebook readable)
yes_no_cols = [c for c in clean.columns
               if clean[c].dtype == 'object' and set(clean[c].dropna().unique()).issubset({'Yes','No'})]
for c in yes_no_cols:
    clean[c] = clean[c].astype('category')

clean.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecur
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	